TP

TradePulse

DISTRIBUTION OPERATIONS PLATFORM

Technical Design Document

Integrated Milestones 2–5 release: authentication and RBAC, retailer catalogue and pricing, order and credit workflows, deterministic AI order capture, and Tally-compatible CSV export.

Repository	https://github.com/Sagar4173/TradePulse
Live application	https://tradepulse-ai-platform.vercel.app/
Health endpoint	https://tradepulse-ai-platform.vercel.app/api/health
Release	Integrated Milestones 2–5
Date	10 November 2026
Status	Deployed and healthy; public OTP delivery blocked

ISO Code Labs Software Engineering Intern Assessment

This document describes the implemented system and its verified release state. It does not claim native Tally XML, production SMS delivery, or passing PostgreSQL integration tests.

Contents

1 Executive summary

2 Product scope and user roles

3 System architecture and design decisions

4 Database model

5 Authentication and authorization

6 Retailer catalogue, pricing, and cart

7 Order management, credit, and stock

8 AI order capture

9 Tally-compatible CSV export

10 UI and UX decisions

11 Testing and release verification

12 Deployment, trade-offs, and conclusion

> **Release snapshot.** The production database has all 9 migrations applied and the deployed health endpoint reports the database as up. The unit gate passes 387 tests across 90 suites. The public deployment cannot complete a sign-in because no real SMS provider is configured; production OTP fails closed by design, and no bypass was added.

Document conventions

“Production” refers to the Vercel deployment and the Neon production branch. “Implemented” means present in the integrated main branch and covered by the release gate. “Blocked” means the verification could not be performed safely with the available infrastructure and is not reported as a pass.

1. Executive summary

TradePulse is an ordering and distribution platform for FMCG distributors and the kirana retailers they serve. This release integrates four milestones into one application:

- **Milestone 2:** phone OTP authentication, database-backed sessions, role-based access control, retailer link review, and administrative user management.
- **Milestone 3:** retailer-specific catalogue, pricing, stock visibility, schemes, a server-authoritative cart, and checkout review.
- **Milestone 4:** order lifecycle, credit exposure and holds, stock reservation, order numbers, and order-desk operations.
- **Milestone 5:** deterministic Hindi/Hinglish/English order capture with retailer-scoped aliases, plus a Tally-compatible CSV export.

1.1 Release outcomes

Outcome	Result
Source control	**Integrated.** M2→M5 history preserved on main; M5 commit fd0280b.
Production database	**Migrated.** 9/9 migrations applied; Prisma reports no schema drift.
Deployment	**Live.** Vercel deployment from main; /api/health returns HTTP 200 with database up.
Automated gate	**Passed.** 387/387 unit tests, lint, typecheck, Prisma validation, and production build.
Public sign-in	**Blocked.** No real SMS provider; production OTP fails closed. No bypass exists.
PostgreSQL integration suite	**Blocked.** No disposable PostgreSQL server; Neon is refused by the harness.

1.2 Key design position

The system is deliberately server-authoritative. The browser holds only a cart intent, never trusted money. Every price, scheme, GST amount, stock position, and credit decision is recalculated or rechecked on the server inside a transaction. Authentication and authorization are enforced in the data-access layer and every server action, not by hiding navigation items.

> **Known release blocker.** The production database has zero user accounts and no SMS adapter. The public application cannot demonstrate an authenticated retailer or order-desk session. The safest legitimate options are to configure a real SMS provider or to run the guarded development OTP flow against a separate disposable database. Development OTP must not be enabled on the production deployment.

2. Product scope and user roles

TradePulse models a distributor's commercial workflow: a retailer builds an order from a personalised catalogue, the server validates price and stock, and the order desk reviews credit and fulfilment. Salesmen and administrators receive scoped operational views.

Role	Primary responsibilities	Data scope
Retailer	Browse catalogue, manage cart, submit orders, view own order history and credit information, use AI capture.	Only the linked retailer record and its orders.
Salesman	View assigned retailers and their orders.	Only retailers in SalesmanRetailerAssignment.
Order Desk	Review incoming orders, approve or reject, manage credit holds, progress lifecycle, export eligible orders to CSV.	All orders and retailers required for order operations.
Admin	Manage users and access, retailers, products, inventory, schemes, audit logs, and order-desk operations.	System-wide, subject to explicit permission checks.

2.1 Core workflows

1. **Sign in:** a verified Indian mobile number receives an OTP; a valid code creates a database session.
2. **Order:** the retailer browses retailer-priced products, adds lines to the cart, reviews server-calculated totals, and submits.
3. **Capture:** the retailer enters Hindi, Hinglish, or English order text, reviews every parsed line, corrects matches, and confirms lines into the normal cart.
4. **Approve:** the order desk checks stock and credit, approves or places the order on credit hold, and progresses it through dispatch and delivery.
5. **Export:** the order desk filters eligible orders by date and downloads a Tally-compatible CSV; the export is audited.

2.2 Non-goals in this release

- No payment gateway, invoicing, or GSTIN capture.
- No native Tally XML integration or Tally import validation.
- No external LLM, image OCR, or voice transcription.
- No van loading, route optimisation, or offline sync.
- No production SMS delivery adapter.

3. System architecture and design decisions

3.1 Runtime topology

Browser → Vercel / Next.js 16 → Server Components + Server Actions → Prisma 7 + pg adapter → Neon PostgreSQL 18

The application runs on Vercel. Prisma CLI operations use the direct Neon endpoint; runtime queries use the pooled DATABASE_URL through the pg driver adapter.

3.2 Layers

Layer	Responsibility
App Router	Public landing page, auth route group, authenticated application shell, health route, and role-scoped pages.
Server Components	Read session and data on the server; render role-scoped pages without shipping authorization decisions to the client.
Server Actions	Re-authorize the actor, validate input, run domain logic, write audit events, and revalidate affected paths.
Domain libraries	Pure and transactional logic for auth, catalogue, cart, schemes, orders, credit, stock, AI capture, and CSV export.
Data access	Prisma Client with a lazy singleton and PostgreSQL driver adapter; explicit selects and role-scoped queries.
Platform	Vercel for deployment; Neon for managed PostgreSQL, branching, and point-in-time recovery.

3.3 Design decisions

Decision	Rationale
Server-authoritative pricing and credit	Prevents client tampering and keeps money logic in one transactional domain layer.
Immutable order snapshots	Price, product, GST, and scheme changes must not rewrite historical orders.
Additive migrations	Production data must survive schema evolution; destructive shortcuts such as db push or migrate reset are prohibited.
Fail-closed authentication	No SMS provider means no production sign-in, rather than an insecure fallback.
Deterministic AI capture	No external model dependency, no prompt-injection surface, predictable behaviour, and no unvalidated product IDs.
CSV export instead of XML	Delivers a usable Tally-compatible interchange format without claiming an unverified native integration.
Database-backed OTP limits	Serverless instances cannot bypass per-phone or per-IP limits through local memory.

3.4 Route map

Route	Access	Purpose
/	Public	Landing page and release overview.
/login	Public	Phone OTP sign-in.
/api/health	Public	Application and database health probe.
/dashboard	Authenticated	Role-specific dashboard.
/catalogue	Retailer	Retailer-priced catalogue, search, filters, schemes, stock.
/cart	Retailer	Server-recalculated cart and checkout review.
/orders, /orders/[id]	Role-scoped	Order history, details, and order-desk actions.
/orders/export	Order Desk, Admin	Tally-compatible CSV export.
/ai-capture	Retailer	Free-form order text capture and line confirmation.
/users, /retailers, /products, /inventory, /schemes, /audit-logs	Admin	Administrative management and audit views.
/account	Authenticated	Account details and sign-out.

4. Database model

The schema is defined in prisma/schema.prisma and applied through 9 reviewed migrations. PostgreSQL is the source of truth for identity, catalogue, stock, credit, orders, and audit history.

4.1 Core entities

Model	Key fields	Purpose
User	id, name, phone, phoneNormalized, role, status, retailerId, phoneVerifiedAt, lastLoginAt	Identity and role. phoneNormalized is unique and format-checked.
Retailer	id, shopName, ownerName, phone, address, creditLimit, isActive	Kirana customer and credit account.
Product	sku, name, description, brand, category, imageUrl, unitLabel, dealerPrice, gstRate, isActive	Catalogue and tax metadata.
RetailerProductPrice	retailerId, productSku, sellingPrice	Current retailer-specific price; one per retailer/SKU.
Inventory	productSku, stockCases, reservedCases, lowStockThreshold	Available stock and reservations.
Scheme	productSku, type, minimumQuantity, freeQuantity, discountPercent, startsAt, endsAt, isActive	BUY_X_GET_Y or percentage discount.
Order	orderNumber, retailerId, createdById, status, source, subtotal, discount, gstAmount, total, credit snapshots, stockState, idempotencyKey	Order header and lifecycle state.
OrderItem	orderId, productSku, productName, unitLabel, quantity, freeQuantity, unitPrice, discount, gstRate, gstAmount, lineTotal, scheme snapshots	Immutable commercial snapshot.
ProductAlias	retailerId, productSku, phrase, normalizedPhrase, confirmationCount, isActive	Retailer-scoped shorthand learned from confirmed capture.
OtpRequestReservation	phone, requestIpHash, status, expiresAt, completedAt	Admission lease created before OTP hashing.
OtpChallenge	phone, codeHash, attempts, maxAttempts, expiresAt, consumedAt, reservationId	Short-lived OTP challenge.
Session	tokenHash, userId, expiresAt, lastSeenAt, revokedAt	Opaque database session.
RetailerLinkRequest	userId, retailerId, status, reviewedById, reviewedAt	Admin-reviewed retailer scope request with preserved history.
SalesmanRetailerAssignment	salesmanId, retailerId, assignedById	Explicit salesman data scope.
AuditEvent	eventType, actorId, entityType, entityId, metadata, createdAt	Activity and security trail.

4.2 Relationships and invariants

- A Retailer has many User accounts, prices, orders, link requests, and salesman assignments.

- A Product has one Inventory, many prices, schemes, order items, and aliases.
- An Order belongs to a retailer and a creator; it has many OrderItem rows. Order history uses restrictive foreign keys or immutable snapshots.
- RetailerLinkRequest preserves reviewed history with ON DELETE RESTRICT for user, retailer, and reviewer.
- Money uses Decimal(12,2) or Decimal(14,2); percentages use Decimal(5,2). Application calculations use integer paise, never floating point.
- CHECK constraints enforce non-negative money and stock, positive quantities, GST/discount ranges, order-number format, credit snapshot non-negativity, and phone format.
- Order totals are GST-inclusive; OrderItem snapshots preserve the product name, unit, price, GST rate, and scheme details used at submission time.

4.3 Migration history

#	Migration	Change
1	20261103210315_init	Initial catalogue, retailer, order, inventory, and scheme schema.
2	20261104010000_data_integrity_checks	CHECK constraints for money, stock, GST, discounts, and quantities.
3	20261105090000_auth_sessions_rbac	User status, sessions, OTP challenges, salesman assignments.
4	20261106090000_user_phone_normalized	Canonical phone backfill, unique index, and format constraint.
5	20261106120000_otp_reservations_retailer_link_reviews	OTP admission ledger and retailer link-request workflow.
6	20261107120000_retailer_link_history_fks	History-preserving indexes and restrictive foreign keys.
7	20261107130000_product_catalogue_metadata	Brand, category, image URL, and category index.
8	20261107140000_order_lifecycle	Order numbers, credit snapshots, stock state, lifecycle constraints.
9	20261109120000_product_alias	Retailer-scoped product aliases for AI capture.

All 9 migrations are applied to the production branch. The normalized-phone migration would abort the transaction on invalid or duplicate existing phones rather than merge or delete users; production had zero users at migration time.

5. Authentication and authorization

5.1 Phone OTP flow

1. The user enters an Indian mobile number. The server normalizes punctuation and prefixes to canonical +91[6-9]XXXXXXXXX.
2. Admission opens a short transaction, acquires PostgreSQL advisory locks for the phone and hashed IP, checks the resend cooldown and rate windows, and creates an OtpRequestReservation.
3. The scrypt hash runs after the admission transaction commits, so expensive hashing never holds a database lock.
4. A six-digit code is generated from a cryptographically secure source and stored only as a salted scrypt hash.
5. Verification checks expiry and attempt count, consumes the code exactly once, invalidates outstanding codes for the phone, and issues a session.
6. First-time verification creates a RETAILER account only. A matching retailer creates a PENDING link request; administrator approval is required before retailer scope is granted.

5.2 Session design

- The browser receives a random opaque token in an HTTP-only, SameSite=Lax, site-wide cookie; the cookie is Secure in production.
- Only a SHA-256 hash of the token is stored in Session.
- The user record is re-read on each request, so role changes, disabling, and revocation take effect immediately.
- Sign-in revokes any token already present in the browser before setting the new token, preventing session fixation.
- Logout revokes the database session and clears the cookie.

5.3 RBAC

The authoritative permission map lives in src/lib/auth/permissions.ts. Navigation is derived from it but is not an authorization mechanism. Protected pages call the session guards, and server actions independently re-authorize the actor.

Capability	Retailer	Salesman	Order Desk	Admin
Dashboard	Yes	Yes	Yes	Yes
Product catalogue	Yes	—	—	—
Own orders	Yes	—	—	—
Assigned-retailer orders	—	Yes	—	—
All orders	—	—	Yes	Yes
Tally CSV export	—	—	Yes	Yes
Own credit information	Yes	—	—	—
All retailer credit	—	—	Yes	Yes
Assigned retailers	—	Yes	—	—

Capability	Retailer	Salesman	Order Desk	Admin
All retailers	—	—	Yes	Yes
View/manage users	—	—	—	Yes
Products, inventory, schemes	—	—	—	Yes
Audit logs	—	—	—	Yes
Account page	Yes	Yes	Yes	Yes

5.4 Retailer link review

A verified phone proves control of a number, not ownership of a business. New verified accounts have no retailer scope. An exact retailer match creates a PENDING RetailerLinkRequest, reviewed by an administrator. Approval uses only the retailer stored on the request; the form cannot substitute another retailer. Approval and rejection are audited. A partial unique index permits at most one PENDING request per user/retailer pair while preserving decided history, and restrictive foreign keys prevent history from being cascade-deleted.

5.5 Production OTP limitation

> **No production SMS provider exists.** createOtpProvider returns an unavailable provider in production. It does not return a code, log a code, or fall back to a development code. AUTH_DEV_OTP_ENABLED is ignored when either NODE_ENV or VERCEL_ENV is production. The development provider additionally requires an allowlisted host and an exact SHA-256 fingerprint of the effective database target. This is intentional fail-closed behaviour, not a missing error path.

The reported limitation is a product/deployment blocker, not an authorization weakness. No universal password or bypass was added to manufacture a demo.

6. Retailer catalogue, pricing, and cart

6.1 Catalogue

The catalogue is retailer-scoped. The candidate query selects active products, the current retailer price, inventory, and active schemes. It never selects dealerPrice for the retailer view. A product without a retailer-specific price is displayed without a purchasable price rather than falling back to dealer cost.

- Search, category filters, sorting, and pagination run on the server over a lightweight candidate set; only the visible page is hydrated with description, image, unit, and GST details.
- Stock is presented as available cases after reservations.
- Active BUY_X_GET_Y and percentage schemes are described against the current time.
- Inactive products and unavailable prices cannot be ordered.

6.2 Cart

The cart cookie stores only owner-bound line intent: SKU and paid quantity, with an optional source marker. It does not store trusted prices or totals. The cart page reloads every product, price, scheme, GST rate, and stock value from the database before rendering.

- The server calculates free quantity from eligible schemes.
- GST is computed from the product snapshot and scheme discount.
- Line and order totals are calculated in integer paise.
- The cart owner is bound to the authenticated user/retailer so a copied cookie cannot be used by another account.

6.3 Checkout review fingerprint

Before submission, the server builds a canonical fingerprint of the authoritative cart: SKU, paid quantity, free quantity, unit price, discount, GST, line total, available stock, and scheme identifiers. The fingerprint travels back with the reviewed form. On submit, the server reloads the cart and compares the fresh fingerprint.

- If price, stock, scheme, or quantity changed, submission is refused with a “review the cart” response.
- The token is not trusted for money; it can only cause another review.
- Any client-supplied totals are ignored.

6.4 Order submission

Submission runs in a serializable transaction with bounded retries. It resolves the actor’s retailer scope, validates the idempotency key, checks for an existing order, recalculates the cart, verifies stock, computes credit exposure, creates the order and items with snapshots, records audit data, and clears the cart only after success.

Duplicate submission with the same retailer-scoped idempotency key returns the existing order instead of creating another one. The duplicate lookup happens before the empty-cart check so a replayed form after cart clearing still resolves to the original order.

7. Order management, credit, and stock

7.1 Lifecycle

PENDING_REVIEW → APPROVED → PROCESSING → READY_FOR_DISPATCH → DISPATCHED → DELIVERED

ON_CREDIT_HOLD is an alternate review state that can be approved, rejected, or cancelled; REJECTED and CANCELLED are terminal outcomes reachable from the permitted source states. DELIVERED is terminal. DRAFT and FULFILLED are legacy values retained for compatibility with earlier order data; the implemented machine defines no transitions out of either.

7.2 Order numbers

Order numbers use the TP-YYYYMMDD-NNNNNN format. The date is the order's creation date in the Asia/Kolkata calendar. A PostgreSQL sequence supplies the six-digit suffix; the backfill migration assigns numbers to pre-existing orders and advances the sequence past them. A CHECK constraint enforces the human-readable format.

7.3 Credit exposure

Credit exposure includes open orders in PENDING_REVIEW, ON_CREDIT_HOLD, APPROVED, PROCESSING, READY_FOR_DISPATCH, DISPATCHED, DELIVERED, and FULFILLED, plus the candidate order's total. ON_CREDIT_HOLD is included because a held order has not been approved but must still count toward exposure. Rejected and cancelled orders never consume credit.

- All credit arithmetic uses integer paise.
- Approval checks projected exposure against the retailer's credit limit.
- If the limit would be exceeded, the order moves to ON_CREDIT_HOLD with a reason and a credit snapshot.
- Re-approval rechecks both credit and stock; a passed credit check with unavailable stock keeps the order on hold with an explanatory reason.
- The order stores credit-limit and outstanding snapshots plus the check timestamp for auditability.

7.4 Stock reservation

- Approval reserves the paid plus free case quantity against Inventory.
- Conditional updates ensure reservations cannot exceed available stock.
- Rejection or cancellation releases an existing reservation; dispatch deducts the reserved quantity from both physical stock and reservations; delivery requires the deducted state.
- stockState records NONE, RESERVED, RELEASED, or DEDUCTED.

7.5 Order desk and audit

Order Desk and administrators see incoming orders, credit status, line details, customer information, and permitted lifecycle actions. Each transition rechecks the actor's permission, the current order state, and relevant invariants inside a transaction, and writes an audit event such as ORDER_APPROVED, ORDER_PROCESSING_STARTED, or ORDER_READY_FOR_DISPATCH. AuditEvent records actor, entity, event type, JSONB metadata, and timestamp for user-access changes, link decisions, bootstrap promotion, lifecycle actions, AI alias changes, and Tally exports. Raw OTPs and session tokens are never written to audit metadata.

8. AI order capture

"AI capture" in this release is a deterministic natural-language pipeline, not a generative model call. It turns a short Hindi, Hinglish, or English order message into reviewable cart lines while keeping the commercial decision on the server.

8.1 Pipeline



8.2 Matching

- Text is normalized for case, punctuation, common Hinglish spellings, and transliteration variants.
- Lines are segmented on punctuation, conjunctions, and quantity cues.
- Quantities and units are parsed, including case-pack hints; invalid or missing quantities produce a clarification state.
- Catalogue candidates are scored using token overlap across name, description, brand, and category, with exact aliases and recent order history as tie-breakers.
- Low-confidence matches remain unresolved and require an explicit product selection; the UI does not silently guess.
- Every selected SKU must exist in the authenticated retailer's current catalogue.

8.3 Aliases

ProductAlias stores a retailer-scoped phrase and its normalized form. The unique key is (retailerId, normalizedPhrase), so one retailer's shorthand cannot change another retailer's meaning. When a retailer corrects a parsed match and confirms it, a new alias may be created, an existing alias may be updated, or an inactive alias may be reactivated. Confirming an already-correct automatic match does not create a duplicate alias. The alias write and its audit event are committed together before the cart cookie is updated.

8.4 Cart and order safety

- Parsing never writes the cart, an alias, or an order.
- Only explicitly confirmed lines are added, and they pass the same catalogue, price, stock, and quantity validation as manual cart lines.
- Confirmed lines land in the normal cart; checkout still requires a separate review and submission.
- Provider output (if a future provider is added) is treated as untrusted: exact keys, bounded text, confidence in [0,1], allowed SKU set, and bounded alternatives.

8.5 Why deterministic

A deterministic matcher avoids external API availability, cost, latency, prompt-injection, and unpredictable SKU selection. It is transparent and testable. The trade-off is narrower language coverage: it does not understand arbitrary slang, images, or voice, and it relies on the retailer's catalogue and confirmed aliases. A future LLM adapter can plug into the provider seam only if its output passes strict validation and the server still recalculates the cart.

9. Tally-compatible CSV export

The Order Desk export at /orders/export produces a flat, line-level CSV intended for Tally-compatible import workflows. It is **not native Tally XML** and has not been validated against a specific Tally version or import template.

9.1 Eligibility

Included	Excluded
APPROVED, PROCESSING, READY_FOR_DISPATCH, DISPATCHED, DELIVERED, FULFILLED	DRAFT, PENDING_REVIEW, ON_CREDIT_HOLD, REJECTED, CANCELLED

Eligibility is enforced in the database query and re-asserted by the pure row builder. A credit-held order has never been approved, and rejected/cancelled orders must never reach Tally.

9.2 Columns

Column	Source / rule
Order Number	Order.orderNumber
Order Date	Creation date in Asia/Kolkata, ISO YYYY-MM-DD.
Customer Name	Current Retailer.shopName.
SKU, Product Name, Unit	Immutable OrderItem snapshot.
Paid Quantity, Free Quantity	OrderItem.quantity, OrderItem.freeQuantity.
Unit Price (INR, GST excl.)	Snapshot unit price, normalized to paise.
Discount (INR)	Line discount snapshot.
Scheme Name, Scheme Type, Scheme Discount (%)	Snapshot scheme fields; blank when absent.
GST Rate (%), GST Amount (INR)	Snapshot rate and amount.
Line Total (INR, GST incl.)	Snapshot line total.
Order Total (INR, GST incl.)	Repeated on every line for the order.

9.3 Safety and limits

- Inclusive Asia/Kolkata date filters; the preview and download are bounded to 500 orders and 5,000 lines.
- The preview reads at most 501 orders and reports 500+ when the cap is exceeded; download is refused.
- UTF-8 with BOM, quoted and escaped cells, and formula-injection neutralization (including values starting with whitespace).
- Deterministic filename and Cache-Control: no-store.
- Each successful download appends an ORDER_TALLY_EXPORTED event on OrderExport with aggregate counts, not row-level customer data.
- Exporting never creates or modifies an order.

9.4 Limitations

- No GSTIN is exported because the schema does not store one.

- Customer name reflects the current retailer shop name, not a historical customer-name snapshot.
- The CSV is Tally-compatible by design, not a certified Tally connector or XML schema.

10. UI and UX decisions

10.1 Design system

- Tailwind CSS 4 with shadcn/ui (Base UI) primitives, a restrained green distribution brand, and a consistent spacing/typography scale.
- Server-rendered pages with partial prerendering and explicit loading, empty, success, error, 401, 403, and 404 states.
- Role-specific navigation derived from the permission map, with a persistent application shell.
- Accessible labels, focus-visible rings, semantic headings, status alerts, and reduced-motion handling for spinners.

10.2 Landing and sign-in

The public landing page presents the integrated release rather than a foundation milestone. It shows the implemented catalogue, cart, credit/order-desk, AI capture, and Tally CSV capabilities, links to sign-in and the health endpoint, and states that production SMS delivery requires a configured provider.

The login card uses a focused single-column layout: phone entry, OTP verification, resend countdown, expiry countdown, and a “change number” action. Development OTP display is gated by the provider and action-layer checks. The mobile layout was corrected to remove horizontal overflow; the card is full-width with a maximum width, the sign-in badge is hidden on very small screens, and the phone input can shrink.

10.3 Retailer and order-desk surfaces

- Catalogue cards expose price, pack size, stock state, and scheme labels without overwhelming the ordering action.
- The cart separates paid and free quantities and shows server-recalculated totals before checkout.
- Order details show lifecycle status, credit state, stock state, line snapshots, and permitted actions.
- The AI capture workbench separates parsing, per-line review/correction, and confirmation; nothing enters the cart until the retailer confirms.
- Tally export separates filter/preview from download and explains empty or over-limit states.

10.4 Responsive verification

Desktop was verified at 1440×1000 and mobile at a 390×844 viewport. The final mobile login geometry is 390px page width with a 358px card and no horizontal overflow. An earlier screenshot appeared clipped because headless Chrome enforces a larger minimum window width; computed layout and document scroll width confirmed the actual responsive layout is correct.

11. Testing and release verification

11.1 Automated gate

Check	Command	Result
Unit tests	npm test	**387 passed / 387; 90 suites; 0 failed**
Lint	npm run lint	**Passed**
Typecheck	npm run typecheck	**Passed**
Prisma schema	npm run db:validate	**Valid**
Production build	npm run build	**Passed**
Migration status	npx prisma migrate status	**9/9 applied; up to date**
Schema drift	prisma migrate diff	**No difference detected**

11.2 Coverage areas

- Phone normalization, OTP expiry/attempts/cooldown/rate limits, provider fail-closed behaviour, development-code suppression, and target-fingerprint guards.
- Session validation, cookie hardening, logout, safe redirects, RBAC, retailer/salesman scope, navigation, access assignment, and bootstrap revocation.
- Catalogue filtering/sorting/pagination, price resolution, scheme eligibility, cart calculations, stock, and checkout review fingerprints.
- Order lifecycle transitions, credit exposure, holds, stock reservation/release, order-number formatting, idempotency, and audit behaviour.
- AI normalization, segmentation, quantity/unit parsing, matching, confidence, aliases, confirmation payload validation, and provider-output validation.
- Tally CSV headers, eligibility, money/percent normalization, escaping, injection protection, limits, filters, and audit metadata.

11.3 Production smoke tests

Check	Observed
/	HTTP 200; release landing page
/login	HTTP 200; sign-in form
/dashboard	HTTP 307 → /login?next=%2Fdashboard
/catalogue	HTTP 307 → /login?next=%2Fcatalogue
/cart	HTTP 307 → /login?next=%2Fcart
/orders	HTTP 307 → /login?next=%2Forders
/orders/export	HTTP 307 → /login?next=%2Forders%2Fexport
/ai-capture	HTTP 307 → /login?next=%2Fai-capture
/api/health	HTTP 200; status ok; application up; database up

Check	Observed
Browser console	No console errors; one transient CSS preload warning observed once on /login
Desktop/mobile	Desktop verified; mobile computed layout has no horizontal overflow

11.4 PostgreSQL integration suite — blocked

> **Not run.** The integration harness requires TRADEPULSE_TEST_DATABASE_ADMIN_URL, creates and drops uniquely named tradepulse_test_* databases, and explicitly refuses Neon hosts, production-looking identifiers, and known production hosts. No disposable PostgreSQL server was available. Running it against Neon or production would violate the harness's safeguards and this release's data-safety rules. No integration-test pass is claimed.

11.5 Production migration verification

- Runtime and direct endpoints resolved to the same effective database target before migration.
- A verified logical backup of all 10 existing tables was created before migration, alongside Neon point-in-time recovery.
- All 7 pending migrations applied with DIRECT_URL; 9/9 total.
- Existing catalogue data preserved: 1 retailer, 3 products, 3 prices, 3 inventory rows, 2 schemes.
- New tables/columns present; Prisma schema diff empty.
- No production seed was run; no users or orders were created.

12. Deployment, trade-offs, and conclusion

12.1 Deployment record

Item	Value
Repository	https://github.com/Sagar4173/TradePulse
M5 feature commit	fd0280bd7f5006f4aa94b59dec5d5ea80ec0c8ce
Integration	Fast-forward of feature/milestone-5-ai-order-capture into main; full M2→M5 history preserved.
Deployment	Existing Vercel tradepulse project, GitHub integration from main.
Live URL	https://tradepulse-ai-platform.vercel.app/
Database	Neon production branch, neondb/public; 9/9 migrations applied.

12.2 Environment names

Required: DATABASE_URL (pooled runtime) and DIRECT_URL (direct Prisma CLI/build). Optional authentication tuning uses AUTH_OTP_* and AUTH_SESSION_TTL_DAYS. Development-only names are AUTH_DEV_OTP_ENABLED, AUTH_DEV_PROVISIONING_ENABLED, TRADEPULSE_DEV_TARGET_HOSTS, and TRADEPULSE_DEV_TARGET_FINGERPRINT. Integration tests require TRADEPULSE_TEST_DATABASE_ADMIN_URL and refuse Neon. AUTH_SMS_PROVIDER, AUTH_SMS_API_URL, AUTH_SMS_API_KEY, and AUTH_SMS_SENDER_ID are reserved names for a future provider and are not implemented. No values or credentials are recorded in this document.

12.3 Migration procedure

```
npx prisma migrate status
npm run db:migrate:deploy
npx prisma migrate status
npx prisma migrate diff --from-config-datasource --to-schema prisma/schema.prisma --exit-code
```

Never use prisma db push, prisma migrate reset, migrate resolve --applied, or destructive SQL against production. Apply migrations before deploying code that depends on them. Recovery uses Neon point-in-time recovery or a verified logical backup; restore into a new branch/database first, validate schema and row counts, then reconnect deliberately.

12.4 Trade-offs and future improvements

Area	Current trade-off	Future improvement
AI capture	Deterministic, text-only, Hindi/Hinglish/English; no generative model, images, or voice.	Add a provider adapter with strict schema validation, confidence thresholds, image OCR, and voice transcription while keeping server-side cart validation.
Production auth	No SMS provider; sign-in fails closed; no public demo session.	Implement a real SMS adapter, configure credentials, add delivery observability, and test verification end-to-end.
Tally integration	Tally-compatible CSV, not native XML; no GSTIN.	Add GSTIN to the retailer model, validate against a real Tally import template, and optionally add XML export.

Area	Current trade-off	Future improvement
Database privileges	Least-privilege runtime roles were not independently inspected in the deployment environment.	Verify the runtime role, replace any owner credential with a least-privilege role, and rotate credentials.
Integration testing	PostgreSQL suite blocked without a disposable server.	Provision an isolated PostgreSQL test service in CI; keep Neon and production excluded.
Cart storage	Owner-bound cookie stores line intent only.	Move to a server-side cart if multi-device continuity or larger carts are required.
Rate limiting	Database-authoritative per-phone/per-IP limits.	Add edge/WAF limiting as defense in depth before high-volume public exposure.
Order operations	Lifecycle and credit implemented; no payment or invoicing.	Add payment reconciliation, invoices, and accounting integration.
Observability	Health endpoint and audit events.	Add structured request logs, metrics, alerting, and export/order dashboards.
Accessibility	Semantic structure, labels, focus states, reduced motion.	Run a formal WCAG audit and assistive-technology testing.

12.5 Release risks

- **BLOCKER:** public production sign-in cannot complete without an SMS provider.
- **IMPORTANT:** no production users/orders means authenticated workflows were not smoke-tested live.
- **IMPORTANT:** the PostgreSQL integration suite is blocked and must be run before broader production reliance.
- **OPTIONAL:** native Tally XML, GSTIN, payments, offline sync, and richer AI modalities.

12.6 Conclusion

TradePulse now ships as one integrated, server-authoritative ordering platform rather than a collection of milestone branches. The release preserves the full M2→M5 history, applies all reviewed migrations to the production Neon database without data loss, deploys from main to the existing Vercel project, and passes the unit, lint, typecheck, schema-validation, and production-build gates.

The strongest engineering choices are the server-side recalculation of every commercial value, immutable order snapshots, database-authoritative OTP rate limiting, history-preserving retailer link review, deterministic AI capture with explicit confirmation, and a bounded, injection-safe Tally-compatible export.

The release is not represented as fully demonstrable in public. Production OTP delivery is intentionally unavailable without a real SMS provider, the production database contains no demo users or orders, and the PostgreSQL integration suite is blocked by the absence of a disposable database. Those limitations are documented rather than hidden or worked around with insecure shortcuts.

> **Release verification statement.** All claims in this document refer to executed checks or inspected production state. Native Tally XML, production SMS delivery, and PostgreSQL integration-test success are explicitly not claimed.